# Property Management

Cool Cats
Cool Cats

# Table of contents

# 1 Introduction

This document outlines the Architectural Specifications for the Property Management System. It includes the selected architectural design, the system's quality requirements, the architectural and design patterns implemented, and an overview of the technologies adopted to develop the system.

# 2 Design Strategy

The team has adopted the Decomposition design strategy to implement the Property Management System.
This approach involves breaking down the system into smaller , manageable subsystems. Each subsystem can be developed, tested, and maintained independently, while still integrating seamlessly into the overall solution.

**Reasons for using Decomposition**

- **Improved understanding of system:** Understanding each smaller part makes it easier for each member to understand the system as a whole.
- **Increased maintainability:** Changes are localised to specific parts, reducing the risk of changes affecting unrelated parts.
- **Faster development**: Each member can be delegated or assigned a part which makes it quicker to develop since each person doesn't need to rely on a component being done first.
- **Better scalability**: Each individual part can be scaled independently based on its needs.

# 3 Quality Requirements

## 3.1 Performance

The PMS (Property Management system) must deliver consistently fast response times to ensure a smooth user experience, particularly when handling property listings, tenant records, and financial transactions. The system should respond to 95% of API requests within 500 milliseconds under normal load conditions. This performance is achieved by using an AWS Application Load Balancer for efficient request distribution, implementing caching on both the frontend and backend, and optimizing PostgreSQL database queries with appropriate indexing and query plans.

## 3.2 Scalability

As the system is expected to support a growing number of property owners, and contractors, scalability is critical. The PMS should be capable of handling a large number of concurrent users while maintaining optimal performance without degradation. This is accomplished by deploying the application in stateless containers that can be automatically scaled using AWS Auto Scaling policies, combined with PostgreSQL read replicas to distribute database read loads efficiently.

## 3.3 Availability

Property management operations require high system uptime to ensure uninterrupted access to key functionalities such as inventory tracking, maintenance requests, budget tracking. The PMS targets 99.5% uptime, monitored via AWS CloudWatch and SLA tracking. Availability is maintained through a Multi-AZ deployment in AWS, automated health checks, and load balancing to reroute traffic during server failures.

## 3.4 Security

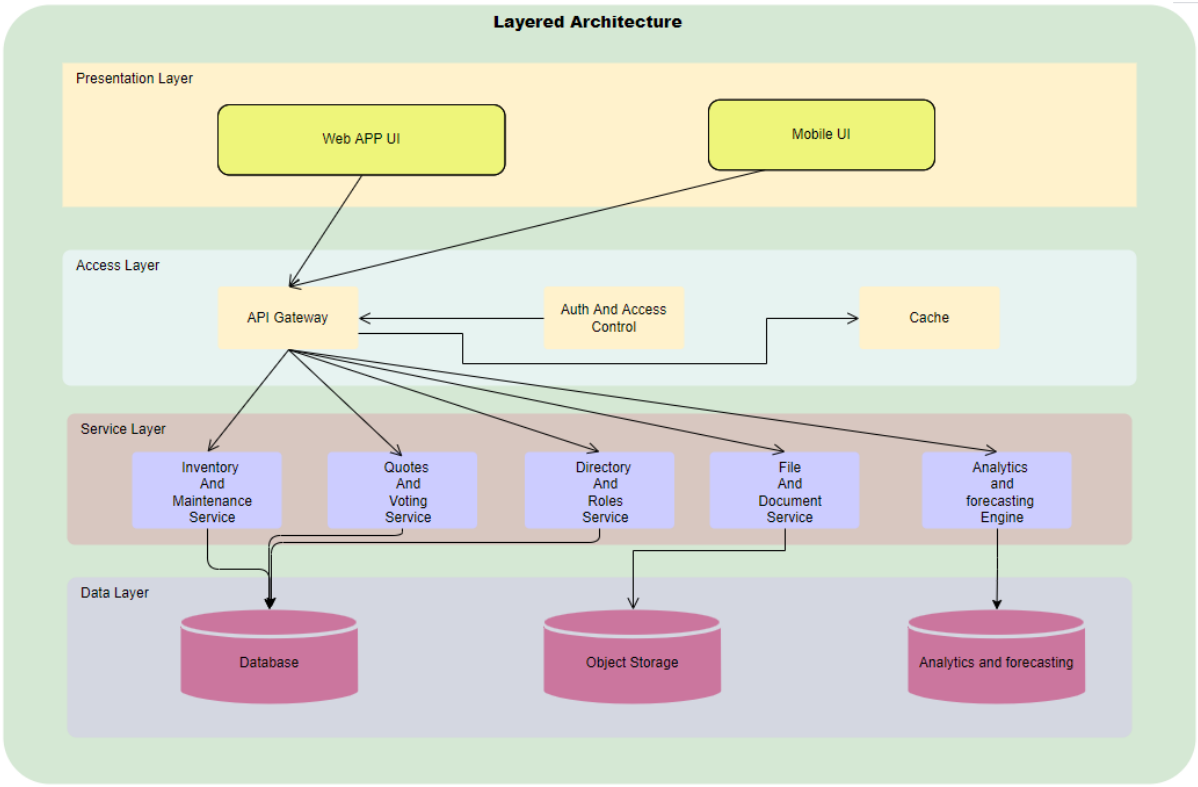
Given the sensitive nature of contractor, expense, and inventory data, the PMS implements essential security measures to ensure data integrity and controlled access. Authentication is managed through AWS Cognito, which provides secure user registration, login, and token-based session management with enforced password policies. Authorization guards are applied across the frontend to restrict access to routes based on user roles, ensuring that Body

Corporate members, Contractors, and Trustees can only perform actions permitted for their roles. In addition, all API endpoints that interact with the database are tested against SQL injection vulnerabilities by simulating common attack patterns to verify that queries remain secure against malicious input.

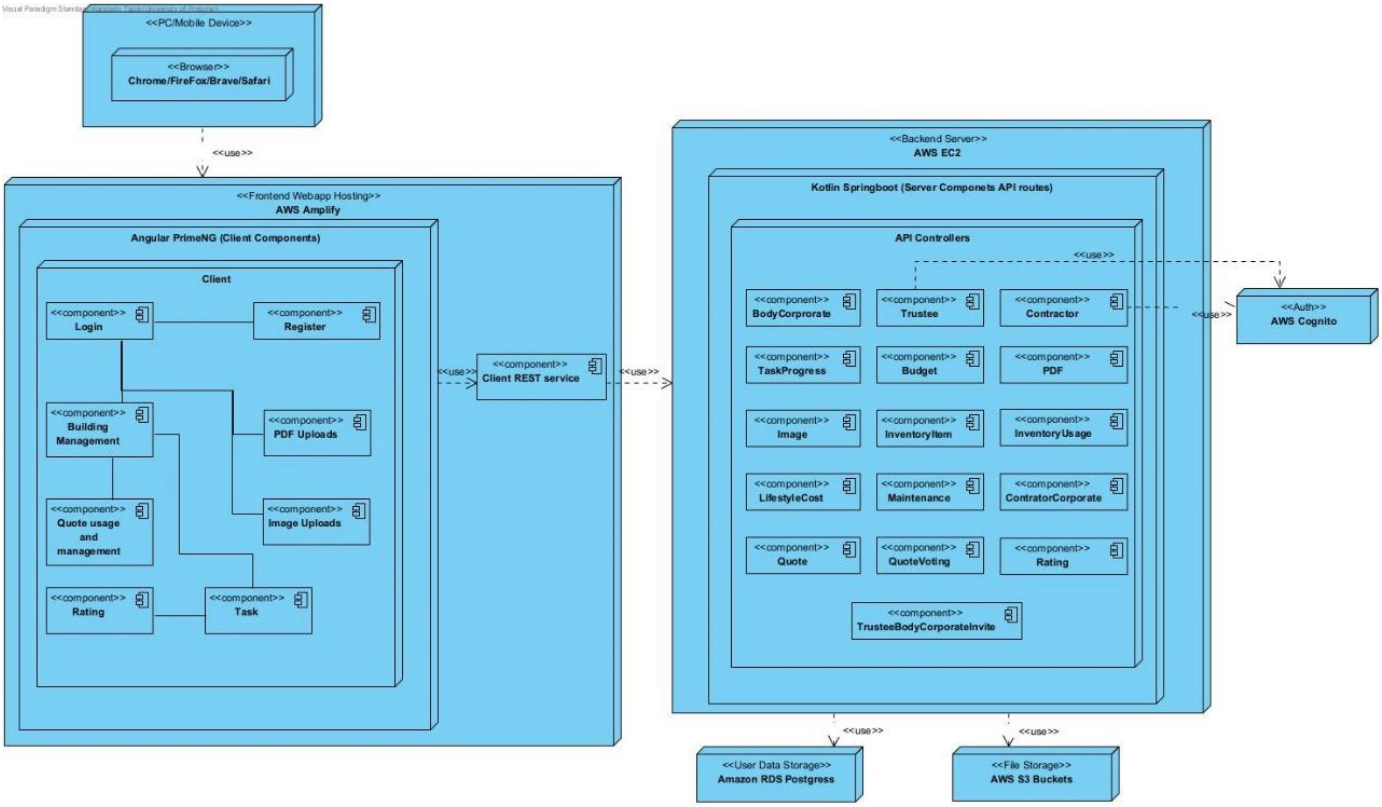
### 3.5 Maintainability

Maintainability ensures that the PMS can evolve with business needs without introducing instability. The system maintains at least 85% unit test coverage and is monitored for code quality using Klint. Modular service design, clear separation of concerns, and a CI/CD pipeline ensure quick, low-risk deployments and bug fixes.

# 4 Architectural Diagram



# 5 Deployment Diagram



# 6 Architectural Patterns

## 6.1 Layered Architecture

The project makes use of a Layered Architecture, which separates concerns across distinct layers within the system. The Presentation Layer consists of an Angular and PrimeNG frontend, served via Ionic to enable hybrid mobile deployment. The Application Layer is implemented using Kotlin Spring Boot services, which contain the core business logic and expose REST endpoints. The Persistence Layer stores structured data in PostgreSQL and media assets in AWS S3. The Integration Layer exposes RESTful APIs to frontend clients and external systems. The Security Layer manages authentication and authorization via AWS Cognito, with session management handled by Spring Security. The Layered Architecture pattern provides clear separation of responsibilities, improves maintainability, and supports secure, modular, and testable development practices.

### 6.1.1 Quality Requirements Addressed

- **Security –** The dedicated Security Layer ensures consistent enforcement of authentication, authorization, and session policies across the system.
- **Scalability –** Layers can be scaled independently, allowing for optimized resource allocation based on system demand.
- **Maintainability –** The structured separation of concerns allows for easier updates, technology changes, and onboarding of new developers.

## 6.2 Microservice Architecture

The system also incorporates a Microservice Architecture at the overall system level, allowing different components to operate as independently deployable services. The Main Backend Microservice, built using Kotlin Spring Boot, handles the core business logic and database operations. The AI Forecasting **Microservice**, developed in Python, runs machine learning models for cost forecasting and anomaly detection. These services communicate securely via REST API calls over HTTPS. Each microservice is deployed independently in Docker containers, which allows them to be scaled, updated, and maintained separately without impacting other components. The Microservice Architecture complements the internal Layered Architecture of the backend, enabling flexibility, modularity, and independent evolution of system components.

### 6.4.1 Quality Requirements Addressed

- **Scalability** - Each microservice can be scaled independently based on its workload, optimizing resource usage and system performance
- **Maintainability –** Independent deployment of services allows for isolated updates, bug fixes, and development without affecting other services.

## 6.3 Model View Controller

The project incorporates the Model View Controller architectural pattern within the Frontend to separate concerns between user interface, interaction logic, and data models. The View consists of Angular components and PrimeNG UI elements, which render the user interface and capture user interactions. The Controller is represented by Angular component classes which handle input validation, orchestrate service calls, and update view state. The Model is represented by interfaces and data transfer objects which define the structure of data exchanged between the frontend and backend. Supporting this separation, angular services encapsulate API calls and business logic, acting as intermediaries between controllers and the backend integration layer.

### 6.3.1 Quality Requirements addressed

- **Maintainability** – Clear separation between view, controller, and model simplifies debugging and testing.

# 7 Technologies Used

## 7.1 Frontend Development

### 7.1.1 Angular

The frontend of the PMS is built using Angular, which provides a structured framework with clear separation of concerns, enabling modular and maintainable UI development. Angular supports component-based architecture, allowing reusable UI elements and faster development cycles.

**Pros**:

- Built-in form handling and routing
- PrimeNG component support
- Enterprise-friendly

**Cons**:

- Heavier learning curve
- Slower builds

### 7.1.2 Ionic

Ionic is used alongside Angular to create hybrid mobile applications. It allows a single codebase to run on multiple platforms, reducing development time and effort while maintaining a consistent user experience across devices.

**Pros**:

- Enables cross-platform mobile deployment with minimal additional work.
- Reuse of Angular components across web and mobile interfaces

**Cons**:

- Hybrid approach may result in slightly larger bundle sizes
- Some device-specific optimizations may be needed for performance

## 7.2 Backend Development

### 7.2.1 Kotlin Spring Boot

The core backend is implemented using Kotlin Spring Boot, which handles the main business logic, REST API endpoints, and database operations. The backend follows a layered architecture (Controller, Service, Repository, Model, DTO) for clear separation of concerns, maintainability, and testability.

**Pros**:

- Strong integration with PostgreSQL and other AWS services.
- Type-safe, robust, and maintainable backend framework.

**Cons:**

- Smaller developer pool compared to Java

### 7.2.2 Python

A separate Python microservice handles AI-powered forecasting and anomaly detection. This service is independently deployable and communicates with the Kotlin backend via REST APIs, enabling specialized processing without affecting the main backend.

**Pros**:

- Ideal for machine learning and data processing workloads.
- Independent deployment allows separate scaling and updates.
- Flexible integration with the main backend through REST APIs.

**Cons:**

- Requires monitoring and orchestration to ensure reliability

### 7.3 Backend Development

Version control is managed using GitHub. ensures collaborative development, code versioning, and change tracking for the Property Management project.

### 7.4 CI-CD

Continuous Integration and Continuous Deployment is implemented using GitHub Actions, leveraging Docker for containerization to ensure consistent environments across development, testing, and deployment.

# 8 Architectural Constraints

### 8.1. Client Defined Constraints

- Must use PrimeNG for the component Library
- Should not implement a custom authentication system, Cognito must be used
- Must make use of free tier AWS

### 8.2. System Constraints

- Design primarily for desktop layout first
- Design a mobile version